Просто DevOps

Что ЕЩЁ надо знать про сети

CIDR, NAT, VLAN, балансировка и прочее

CIDR и подсети

Просто DevOps

IP-адрес: 192.168.1.0/24 — разбивка на биты

11000000 . 10101000 . 00000001 . 00000000

11111111.11111111.11111111.00000000 — маска /24

Часть сети — 24 бита (фиксированы маской)	Часть хостов — 8 бит

Адрес сети	Первый хост	Последний хост	Broadcast
192.168.1.0	192.168.1.1	192.168.1.254	192.168.1.255

Формула

Хостов = 2^(32 - маска) - 2

2^(32−24) − 2 = 2^8 − 2 = 256 − 2 = 254 хоста

−2: адрес сети и broadcast не используются под хосты

IP-адрес 192.168.1.0/24: 24 бита — сеть, 8 бит — хосты. Итого 254 рабочих адреса.

Шпаргалка по маскам

Просто DevOps

```
# Маска  Хостов       Применение
/32      1            Конкретный IP (Security Group, /32 route)
/30      2            Point-to-point линк между роутерами
/28      14           Маленькая подсеть
/27      30           Небольшой кластер
/26      62           Средняя подсеть
/25      126          Половина класса C
/24      254          Типичная локальная сеть
/22      1 022        Несколько филиалов
/16      65 534       Большая корп. сеть
/8       16 777 214   Мегаблок (10.0.0.0/8 — RFC 1918)
```

Формула: 2^(32−маска)−2. /32 — один хост, /0 — весь интернет.

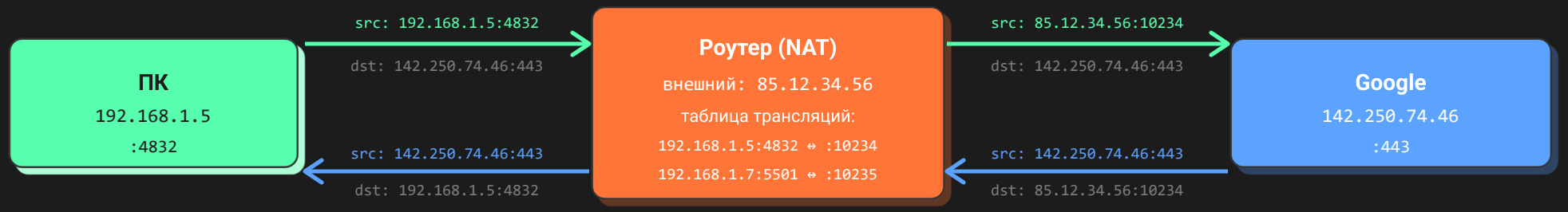
Приватные и публичные IP

Просто DevOps

- 10.0.0.0/8 — корпоративные сети
- 172.16.0.0/12 — 1 млн адресов (например, Docker default bridge — 172.17.0.0/16)
- 192.168.0.0/16 — 65 тыс. адресов — домашний или офисный интернет
- Эти диапазоны не маршрутизируются в интернете, поэтому нужен NAT (RFC 1918)
- Всё остальное — это ПУБЛИЧНЫЕ IP, уникальны в мире, выдаются провайдером

Как работает NAT

Просто DevOps



Исходящий пакет (SNAT)

src: 192.168.1.5:4832

↓ роутер подменяет src

src: 85.12.34.56:10234

dst остается: 142.250.74.46:443

Ответный пакет (обратная трансляция)

dst: 85.12.34.56:10234

↓ роутер ищет в таблице, подменяет dst

dst: 192.168.1.5:4832

src остается: 142.250.74.46:443

NAT подменяет src-адрес при выходе и восстанавливает dst при возврате, используя таблицу трансляций.

SNAT, DNAT, Masquerade

Просто DevOps

- SNAT (Source NAT) — это подмена адреса отправителя:Он подменяет все хосты локальной сети и они выходят в интернет через один IP
- DNAT (Destination NAT) — это подмена адреса получателя: проброс порта, kubectl port-forward. Пакет на 85.x.x.x:8080 → перенаправляется на 10.0.1.5:80
- Masquerade — SNAT с автоопределением внешнего IP интерфейса. Нужно при динамическом IP от провайдера (DHCP)
- Kubernetes kube-proxy — это DNAT. ClusterIP реализован через iptables DNAT-правила

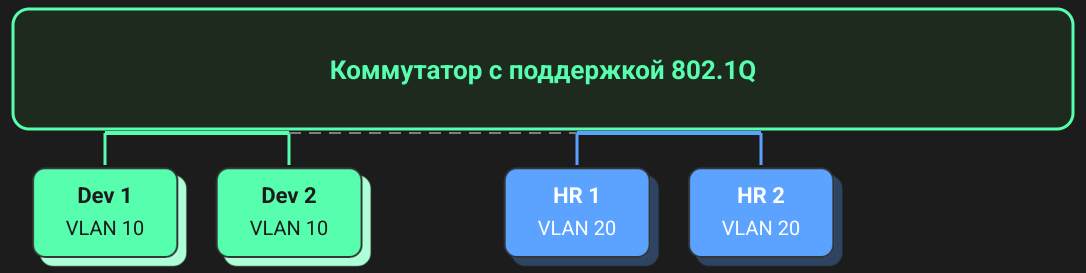
VLAN: изоляция на L2

Просто DevOps

БЕЗ VLAN — все видят всех

Dev видит трафик HR — уязвимость!

С VLAN — изолированные домены



Тег 802.1Q в Ethernet-кадре

`VLAN ID: 10 или 20 (12 бит)`

Коммутатор читает тег, направляет только в нужные порты

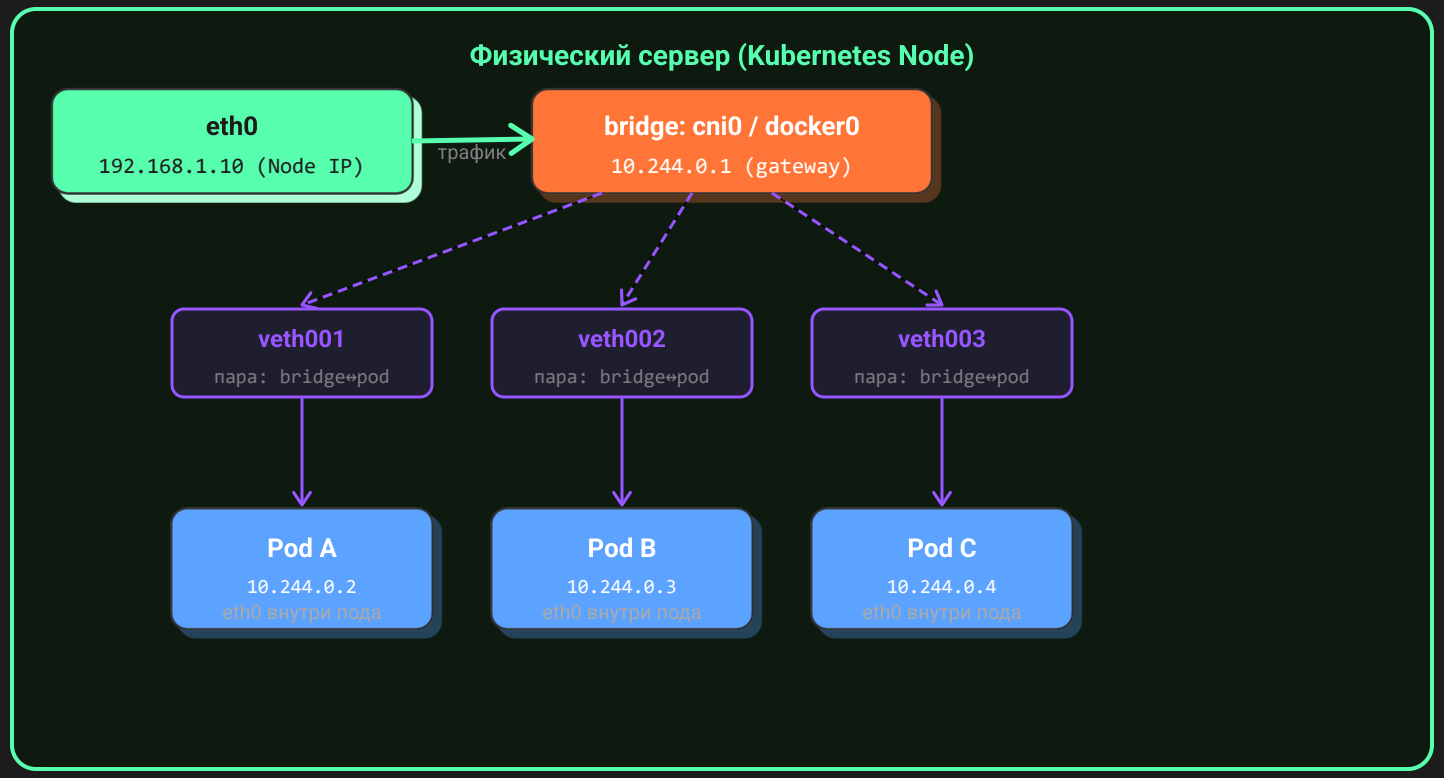
Применение VLAN в DevOps

- Prod / Staging / Dev — изолированные сети
- Management VLAN — отдельный доступ к серверам
- Storage VLAN — трафик СХД не мешает бизнес-трафику
- Безопасность: L2-изоляция без физического разделения

802.1Q тег вставляется в Ethernet-кадр. Один коммутатор = несколько изолированных L2-сегментов.

Виртуальные сети в Kubernetes

Просто DevOps



CNI-плагины

Flannel
простой overlay через VXLAN

Calico
BGP-маршрутизация, NetworkPolicy

Cilium
eBPF, L7 policy, observability

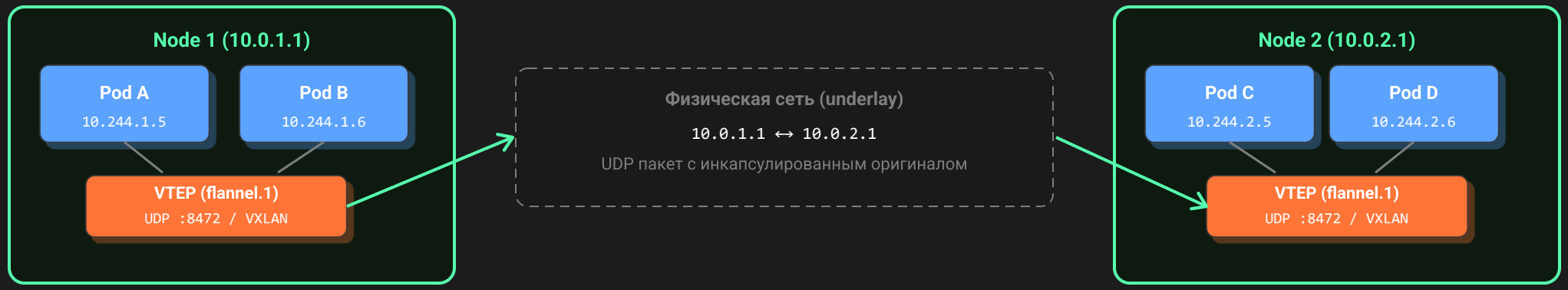
CNI отвечает за:

- Создание veth-пары для пода
- Подключение к bridge
- Выдача IP из podCIDR
- Маршрутизация между нодами

veth-пара — виртуальный кабель: один конец в поде, другой в bridge на ноде. CNI-плагин управляет этой топологией.

Overlay-сети и VXLAN

Просто DevOps



Инкапсуляция VXLAN: пакет пода оборачивается в UDP

Outer	Outer	UDP	VXLAN	Оригинальный Ethernet-фрейм пода (Inner)
ETH Node MACs	IP Node IPs	:8472 VTEP порт	VNI 8 байт заголовок	ETH | IP: 10.244.1.5→10.244.2.5 | TCP | Данные пода Pod A видит только этот слой — для него всё прозрачно

VTEP на Node 1 инкапсулирует → физическая сеть доставляет UDP → VTEP на Node 2 декапсулирует

VXLAN инкапсулирует L2-фреймы подов в UDP. Поды в разных подсетях видят друг друга как в одной сети.

ARP и DHCP

Просто DevOps

- ARP (Address Resolution Protocol): хост знает IP, но нужен MAC для отправки Ethernet-кадра. Отправляет broadcast «У кого IP X?», получатель отвечает своим MAC. Кешируется в ARP-таблице (ip neigh). Протокол L2
- DHCP DORA: Discover (broadcast) → Offer (сервер предлагает IP) → Request (клиент принимает) → ACK (подтверждение)
- DHCP выдаёт не только IP, а ещё маску подсети, шлюз по умолчанию, DNS-серверы, время аренды (lease). Протокол L3

Reverse Proxy и балансировка

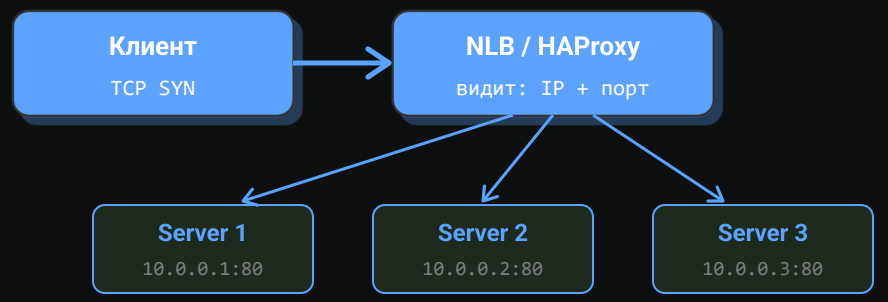
Просто DevOps

- SSL-терминация: TLS-шифрование обрабатывается на прокси — бэкенды работают по plain HTTP внутри сети
- Балансировка нагрузки: запросы распределяются между несколькими инстансами бэкенда
- Единая точка входа, то есть один домен/IP снаружи → множество микросервисов внутри по URL-пути

L4 vs L7 балансировка

Просто DevOps

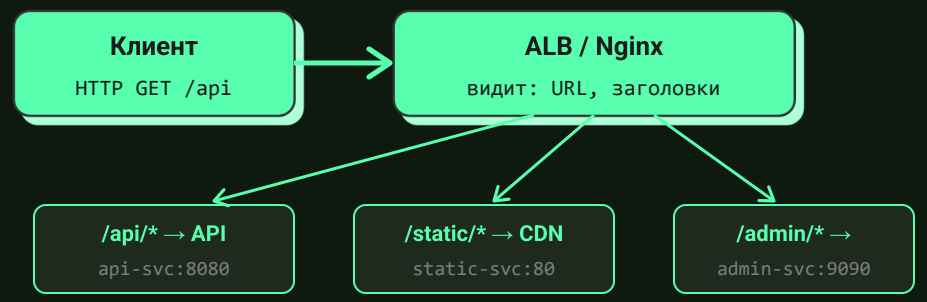
L4 — Транспортный уровень



L4 видит и решает:

- IP src/dst, TCP/UDP порты
- Алгоритмы: Round Robin, Least Conn
- Пример - HAProxy TCP
- Не видит HTTP-заголовки и URL
- Быстрее, меньше накладных расходов

L7 — Прикладной уровень



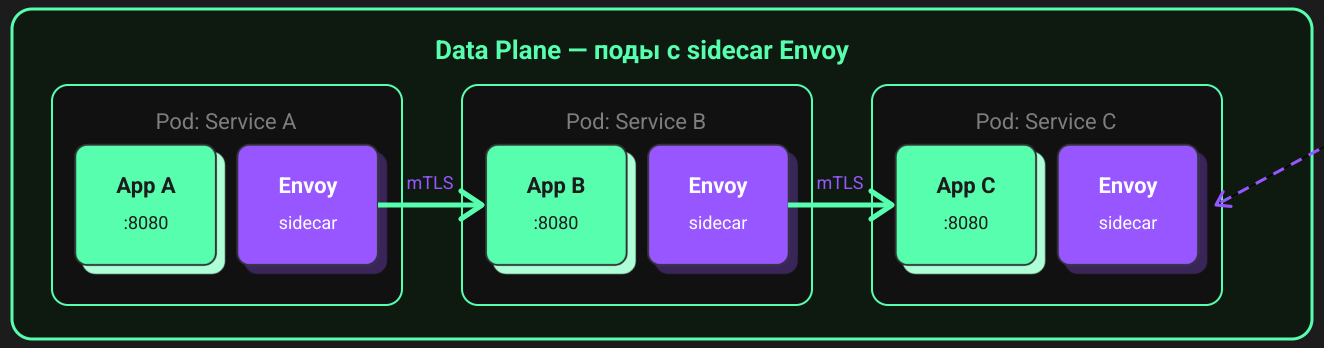
L7 видит и решает:

- HTTP/HTTPS, URL-путь, заголовки, cookie
- Маршрутизация по контенту (content routing)
- Примеры: Nginx, Traefik, Envoy
- SSL-терминация, сжатие, кеш
- Медленнее L4, но умнее

L4 балансирует по IP/порту без понимания контента. L7 маршрутизирует по URL/заголовкам — каждый запрос к нужному сервису.

Service Mesh

Просто DevOps



Control Plane

Istiod

Pilot · Citadel · Galley

Istiod управляет всеми Envoy-прокси:

- Раздаёт конфиги (xDS API)
- Выдаёт TLS-сертификаты (mTLS)
- Политики трафика (retry, timeout)
- Circuit breaker

Envoy собирает метрики:

- Latency p50/p95/p99
- Error rate (4xx/5xx)
- Request throughput (RPS)
- Distributed tracing (Jaeger)

Что даёт Service Mesh

- mTLS автоматически между ВСЕМИ сервисами (zero-trust)
- Retry и timeout без изменения кода приложения

Падение одного сервиса не рушит всё

- Метрики и трейсы из коробки (Prometheus, Jaeger)
- Traffic shifting: canary 10% → 90% без Ingress-правок

Sidecar Envoy перехватывает весь трафик пода. Istiod (control plane) централизованно управляет политиками и сертификатами.

Утилиты для дебага сети

Просто DevOps

```
# L3: маршрутизация
ip addr show                    # IP-адреса интерфейсов
ip route show                   # таблица маршрутов
ping 8.8.8.8                    # доступность хоста
traceroute 8.8.8.8              # путь до хоста (TTL)
mtr 8.8.8.8                     # realtime traceroute

# L4: транспортный уровень
ss -tulnp                       # открытые порты + процессы
nc -zv host 443                 # проверка TCP-соединения

# L7: прикладной уровень
dig @8.8.8.8 example.com        # DNS-запрос
curl -v https://example.com     # HTTP с заголовками
tcpdump -i eth0 port 80         # перехват трафика
wireshark                       # GUI-анализатор пакетов
```

Дебаг сверху вниз: сначала L7 (curl), потом L4 (ss), L3 (ping/route), L2 (ip neigh). Сужай проблему до уровня.